

Workshop Registration Service

Full Stack Challenge | 3 hours

BUSINESS CONTEXT

Who is this for?

A community training centre with three locations and around 15 staff. It runs short workshops (pottery, coding, fitness and more) with a limited number of seats. The front desk takes registrations by phone and in person, a programme manager schedules the workshops, and an administrator looks after staff accounts. The team is non-technical. They need the system to just work, no training required.

The problem they're solving

Registrations are handled over the phone and written into a shared spreadsheet. Workshops get overbooked, and when someone cancels, nobody frees up the seat or records who did it.

"Last Saturday 24 people turned up for a workshop with 20 seats. Two of us had each promised the last seats on the phone at the same time."

WHAT THE CLIENT NEEDS

1. Staff access & permissions

Three types of users, each with different levels of access. There is no public signup: the first Admin account is seeded, and Admins create every other account.

What needs doing	Admin	Manager	Staff
Create user accounts & set roles	✓ Yes	No	No
Add & edit workshops	No	✓ Yes	No
Register & cancel attendees	No	✓ Yes	✓ Yes
View workshops, registrations & history	No	✓ Yes	✓ Yes

Anything not marked must be refused by the backend, not just hidden in the interface.

2. Workshop catalogue

The client shared what they currently track in their spreadsheet:

- Workshop code
- Title
- Instructor
- Date & time
- Capacity (number of seats)
- Current status

"If you think there's anything else worth tracking, go ahead. You know better than us."

3. Registrations

Front desk staff register attendees for a workshop. An attendee is just a name and an email address typed in by staff; attendees do not have accounts. When a registration is cancelled the seat is freed, but the record is never deleted. The full history of every registration, including cancellations, and who registered or cancelled it and when, should always be accessible.

The one rule the client cares about most: a workshop can never hold more active registrations than its capacity.

"Two of us promising the last seat at the same time is exactly what we are trying to stop."

"Saturday mornings are chaos. Several of us will be registering people for the same popular workshop at the same moment."

4. Finding workshops

Staff need to quickly find workshops by date range, status, or seats still available.

"I just want to see which workshops this week still have seats, without scrolling through everything."

TECHNICAL NOTES

What you're building

This is a full stack challenge. You are responsible for both the backend API and the frontend interface. They should work together as a connected, working application.

Time

You have 3 hours. You are not expected to polish everything. If you run out of time, finish the core well and tell us what you skipped. Suggested priority:

- Access control enforced on the backend
- The capacity rule and registration history
- A frontend that works end to end
- Finding workshops
- Everything else

Stack

Your choice: use what you know best. Be ready to justify your decisions in the written document.

Bonus (optional)

The following are not required but will demonstrate extra initiative:

- Audit trail: a history of other changes (workshop edits, account and role changes), recording who did what and when
- Waitlist: when a workshop is full, queue attendees and offer the seat to the next in line when one is freed

DELIVERABLES

- **Source code:** GitHub repo preferred, or zipped via email. One repo containing both backend and frontend is fine.
- **Live deployment (optional):** any free hosting is fine.
- **Setup instructions:** clear enough that we can run both frontend and backend locally without asking you anything. Include a seeded Admin login (dev-only credentials in the README are fine) and a few sample workshops so we can try it straight away.
- **A short written document (one page):** stack choices and why, design decisions, trade-offs, assumptions made, how you prevent over-registration, and anything you skipped.

WHAT WE LOOK FOR

There's no single correct answer. We care about how you think across the full stack: how you design the API, structure the frontend, and enforce access control, and whether the capacity rule still holds when requests arrive at the same moment.

A clean, connected, working application will always score higher than a feature-heavy but broken one.